

Introduction to FLR

a4a Training Course — Module 1.1

Danai Mantopoulou Palouka & Alessandro Ligas

2026-09-13

The problem FLR was built to solve

Every stock assessment group used to write its own software.

- Methods that could not be compared
- Results that could not be reproduced
- Code that died with the person who wrote it



The consequence

You could not test whether a management strategy was **robust**, because you could not connect the pieces.

What FLR is

*“A development effort directed towards the evaluation of fisheries management strategies. The overall goal is to develop a **common framework** to facilitate collaboration within and across disciplines.”*

FLR — *Fisheries Library for R*

- A **collection of R packages**, not a single program
- A **common set of objects** every package understands
- An **open source** project, still actively developed

Kell *et al.* (2007) *FLR: an open-source framework for the evaluation and development of management strategies*. ICES J. Mar. Sci. 64(4): 640–646.

The design principles

Principle	What it means for you
Common currency	Every package speaks the same objects
Object-oriented	Data and metadata travel together
Extensible	New methods plug into the same structures
Open source	You can read, check and change the code

Why this helps you in practice

- Your stock object works with **any** assessment method in FLR
- Swap a4a for XSA or SAM — **the data does not change**
- The output of the assessment feeds **straight into** reference points and forecasts, with no re-formatting
- Plotting, summarising and subsetting work the **same way** everywhere



The practical payoff

You prepare your data **once**, on Day 1. Everything else this week reuses that same object.

Area	Packages
Core objects & plotting	FLCore, ggplotFL
Stock assessment	FLa4a, FLXSA, FLSAM
Reference points & advice	FLBRP
Forecasting & projection	FLasher, FLash
Life history & biology	FLife
Management Strategy Evaluation (MSE)	mse, mseviz, FLFishery
Diagnostics	FLa4a, a4adiags

You do **not** need to know these. You need to know the six we use.

The six we use this week

Package	What it gives us	When
FLCore	The objects: FLQuant, FLStock, FLIndex	Day 1
ggplotFL	Plot methods for FLR objects	Day 1
FLa4a	The assessment model itself, sca()	Days 2–4
a4adiags	Retrospective & hindcast diagnostics	Day 4
FLBRP	Reference points	Day 5
FLasher	Short-term forecasts	Day 5

These are exactly the packages your setup check just loaded.

Two habits worth picking up now

Use accessors

```
catch(stk)           # yes  
stk@catch            # no  
catch(stk) <- newdata # checks the object stays valid
```

Every class has a plural — FLStock → FLStocks, FLIndex → FLIndices,
FLQuant → FLQuants

Getting help

```
help("FLStock")           # class documentation
?sca                      # a method
methods(class = "FLStock") # what can I do with this?
showClass("FLStock")
```

- FLa4a documentation: <https://flrproject.org/FLa4a/>
- And: we are around all week

Looking inside a method

```
methods(class = "FLStock")    # what can I do with this object?  
showMethods("plot")          # which classes does plot() handle?
```

plot(ple4) dispatches on **two** arguments — x="FLStock", y="missing":

```
getMethod("plot", signature = list("FLStock", "missing"))  
  
.local <- function (x, metrics = list(Rec = rec, SSB = ssb,  
                                     Catch = catch, F = fbar), na.rm = TRUE, ...)
```

That default metrics list is **why** plot(ple4) gives you those four panels — and how you ask for different ones.

i Why S4 and not plain R?

FLR uses **S4** classes rather than ordinary lists.

- **Slots are typed** — an FLStock cannot hold the wrong thing
- **Validity is checked** on creation and on replacement
- **Methods dispatch on class** — `plot()` knows what it was handed

The cost is a steeper learning curve. The benefit is that a malformed stock object fails loudly on Day 1, rather than silently producing a wrong assessment on Day 3.

Summary

- FLR is a **common framework**, not one program
- Its value is **interoperability**: prepare data once, reuse all week
- We use **six** packages; FLCore is the foundation
- Use **accessors**, and watch the **plurals**

Next: **Module 1.2** — FLQuant, the building block everything is made of.